

Ya2yOS 内核设计文档

基于 Rust 的 Linux 兼容宏内核



RISC-V64 / LoongArch64 双架构支持

目录

1 项目概述	3
1.1 设计目标	3
1.2 总体架构	3
2 启动与系统调用	4
3 进程与调度	4
3.1 生命周期	5
3.2 调度器	5
4 内存管理	5
4.1 页表与地址空间	6
4.2 COW 与按需分页	6
5 信号机制	7
6 文件系统	7
6.1 VFS 分层	7
6.2 ext4 集成与兼容对象	8
7 网络模块	8
8 IO 设备	8
9 项目特色与边界	8
9.1 特色	8
9.2 当前边界	9
10 总结与展望	9

1 项目概述

Ya2yOS 是一个基于 Rust 语言开发的宏内核操作系统，以 TatlinOS 为基础持续演进，面向全国大学生系统能力大赛内核实现赛道的 Linux 用户态兼容目标。内核支持 RISC-V64 与 LoongArch64 两种架构，在统一的系统调用、进程、内存、文件、网络和设备抽象之上适配不同启动方式、页表机制和 VirtIO 传输方式。

项目的核心目标不是构建一个最小教学内核，而是提供能够运行真实用户态程序的综合内核：支持 BusyBox、glibc/musl libc-test、LTP 子集、lmbench、iozone 等负载，并围绕测试反馈持续补齐 Linux ABI 细节。

1.1 设计目标

目标	说明
Linux syscall 兼容	实现 200+ 个系统调用，覆盖进程、内存、文件、网络、信号、同步、时间和 IO 多路复用。
完整进程模型	支持 fork/clone/clone3、execve、exit、wait4/waitid、线程组、父子关系和进程组基础语义。
现代虚拟内存	支持 SV39/LA64 页表、mmap/munmap/mprotect/brk、COW、按需分页和共享映射。
POSIX 信号机制	支持 sigaction、信号屏蔽、信号帧、sigreturn、SIGCHLD、停止/继续和常见实时信号路径。
真实文件系统	通过 lwext4_rust 接入 ext4，并以 VFS 统一普通文件、设备、管道、socket、epoll/eventfd 等对象。
网络与设备	基于 smoltcp 支持 TCP/UDP/Unix socket，基于 VirtIO 支持块设备和网络设备。

1.2 总体架构

Ya2yOS 采用宏内核结构，核心子系统均运行在内核态并共享内核地址空间。os/src 下的主要模块包括：

- arch/：RISC-V64 与 LoongArch64 架构适配、上下文切换、页表和中断入口。
- trap/ 与 syscall/：陷入处理、系统调用分发和各类 syscall 实现。
- task/：任务控制块、进程对象、调度器、futex 和线程生命周期管理。
- mm/：地址空间、页表、物理帧分配、mmap、COW、缺页处理和用户指针拷贝。
- fs/：VFS、ext4 适配、fd 表、设备文件、proc 兼容文件、epoll/eventfd/inotify/mqueue。
- net/：socket 抽象、TCP/UDP/Unix socket、smoltcp 服务和路由设备。
- drivers/：VirtIO 块设备、VirtIO 网络设备、DMA HAL、控制台与设备抽象。
- signal/、timer/、sync/：信号、定时器和内核同步原语。

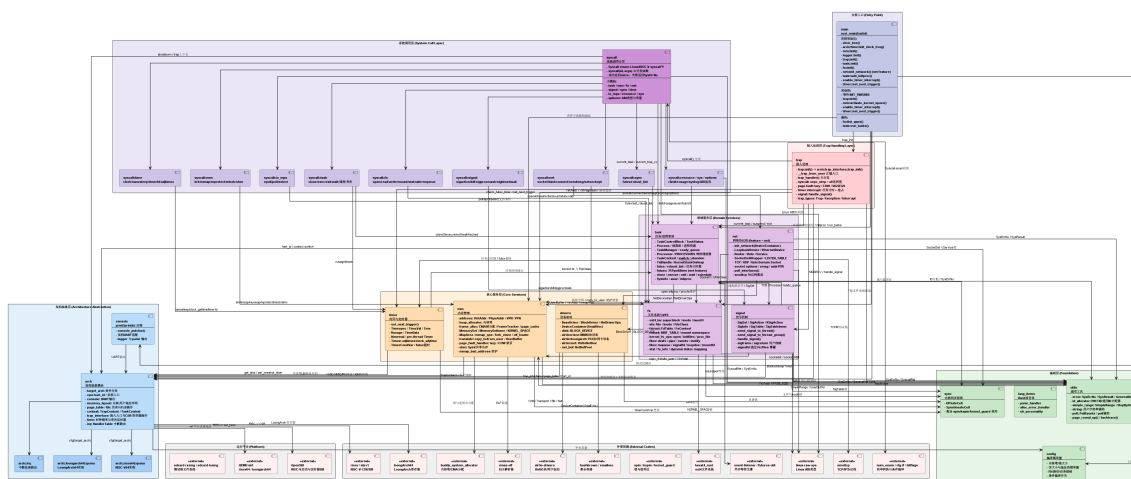


图 1 Ya2yOS 模块依赖关系

2 启动与系统调用

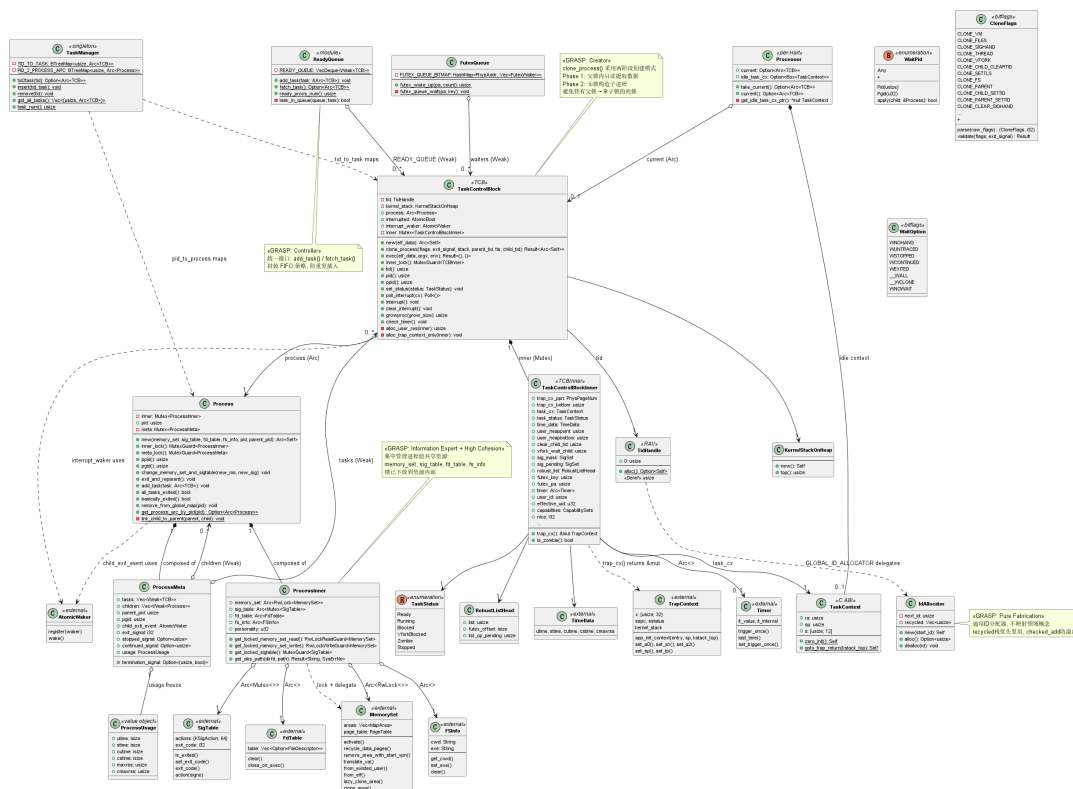
内核由 OpenSBI 或 LoongArch 平台固件引导进入 `_start`，随后完成 BSS 清零、栈初始化、页表启用和 `rust_main` 初始化。第一个 Hart 初始化内存管理、日志、陷入向量、任务管理、文件系统、网络设备和 `initproc`；其他 Hart 等待全局初始化完成后加入调度循环。

系统调用由用户态 `ecall` 或对应架构陷入指令进入内核，参数通过 ABI 寄存器传入。内核将 `syscall id` 转换为枚举后进行 `match` 分发，返回值沿用 Linux 负 `errno` 约定。

类别	代表接口
进程管理	<code>clone</code> , <code>clone3</code> , <code>execve</code> , <code>exit</code> , <code>exit_group</code> , <code>wait4</code> , <code>waitid</code> , <code>set_tid_address</code>
内存管理	<code>mmap</code> , <code>munmap</code> , <code>mprotect</code> , <code>brk</code> , <code>mremap</code> , <code>shmget</code> , <code>shmat</code> , <code>shmctl</code>
文件系统	<code>openat</code> , <code>read</code> , <code>write</code> , <code>getdents64</code> , <code>statx</code> , <code>mount</code> , <code>renameat2</code> , <code>fallocate</code>
信号机制	<code>rt_sigaction</code> , <code>rt_sigprocmask</code> , <code>kill</code> , <code>tgkill</code> , <code>sigsuspend</code> , <code>rt_sigreturn</code>
网络	<code>socket</code> , <code>bind</code> , <code>listen</code> , <code>accept</code> , <code>connect</code> , <code>sendmsg</code> , <code>recvmsg</code> , <code>setsockopt</code>
同步与事件	<code>futex</code> , <code>epoll_create1</code> , <code>epoll_ctl</code> , <code>epoll_pwait</code> , <code>eventfd2</code> , <code>ppoll</code> , <code>pselect6</code>

3 进程与调度

Ya2yOS 采用 Linux 风格的轻量级进程模型。`Process` 代表资源集合，持有地址空间、文件描述符表、信号表和文件系统上下文；`TaskControlBlock` 代表可调度执行单元，持有寄存器上下文、内核栈、线程私有信号状态、`futex` 状态和计时信息。同一进程内的多个线程通过 `Arc<Process>` 共享进程资源。



5

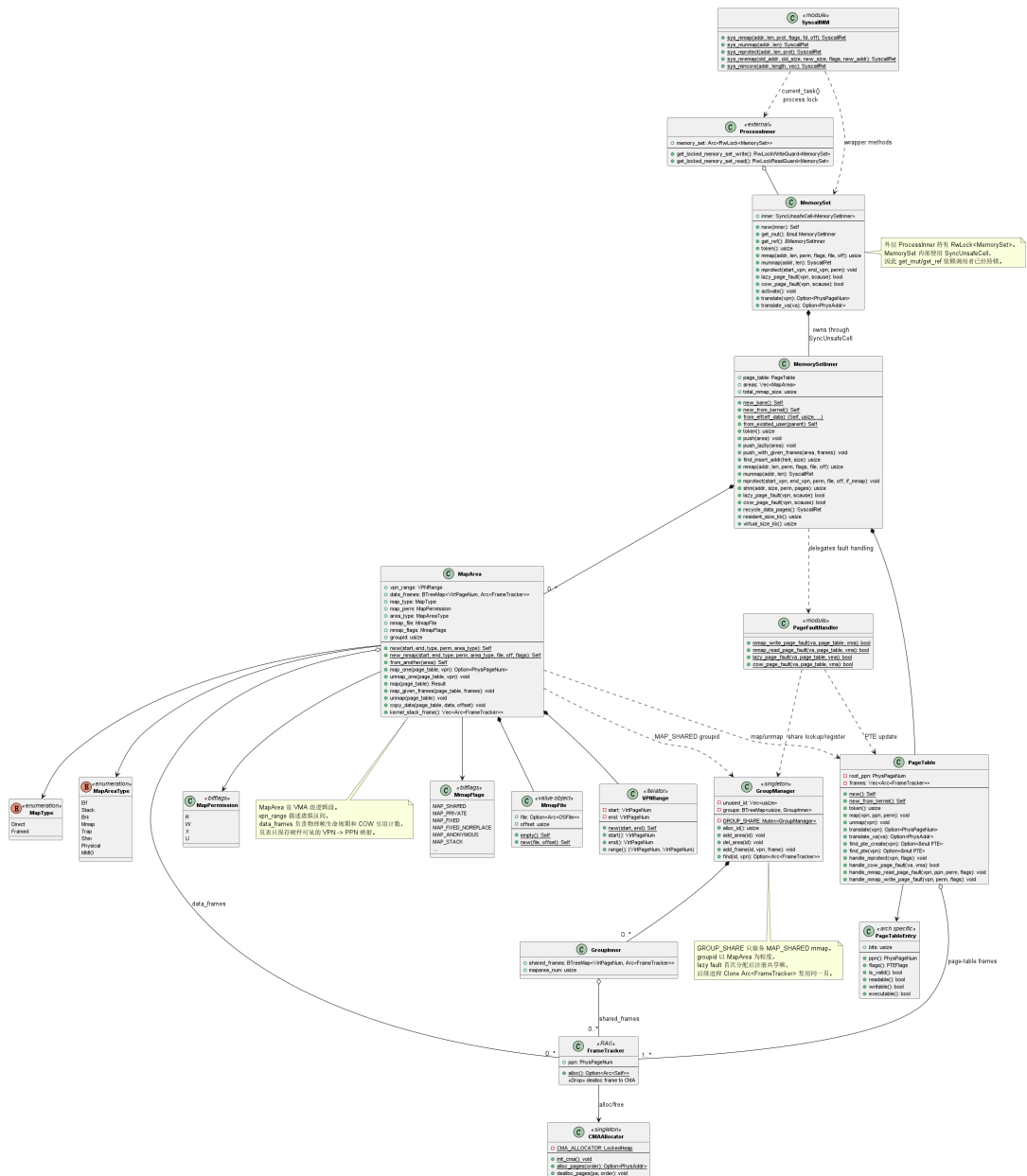


图 3 内存管理核心结构

4.1 页表与地址空间

RISC-V64 使用 SV39 三级页表，LoongArch64 使用对应 LA64 页表机制。页表项权限由架构适配层转换为统一的 MapPermission。用户页表创建时拷贝内核高地址映射，使陷入内核后无需立即切换页表。

MemorySet 维护一组不重叠的 MapArea，每个区域记录起止 VPN、映射类型、权限和后端来源。mmap、munmap、mprotect 等操作通过拆分、合并、改权限和解映射维护 VMA 结构。

4.2 COW 与按需分页

fork 时父子进程共享可写页：页表清除写权限并设置软件 COW 标志。写入触发缺页后，内核检查物理页引用计数；若独占则直接恢复写权限，若共享则分配新页并复制内容。mmap 文件映射、brk 堆和 lazy allocation 也通过缺页处理路径完成实际物理页分配。

缺页处理路径区分匿名页、COW 页、文件映射读写页和非法访问。无法处理的用户态页错误转化为 SIGSEGV，内核态异常则触发 panic。

5 信号机制

信号机制实现 Linux/POSIX 兼容的异步事件通知。进程级 SigTable 保存每个信号的处理动作，线程级状态保存 pending set、mask 和中断标记。信号发送可以面向线程组、单个 TID 或全局进程集合。

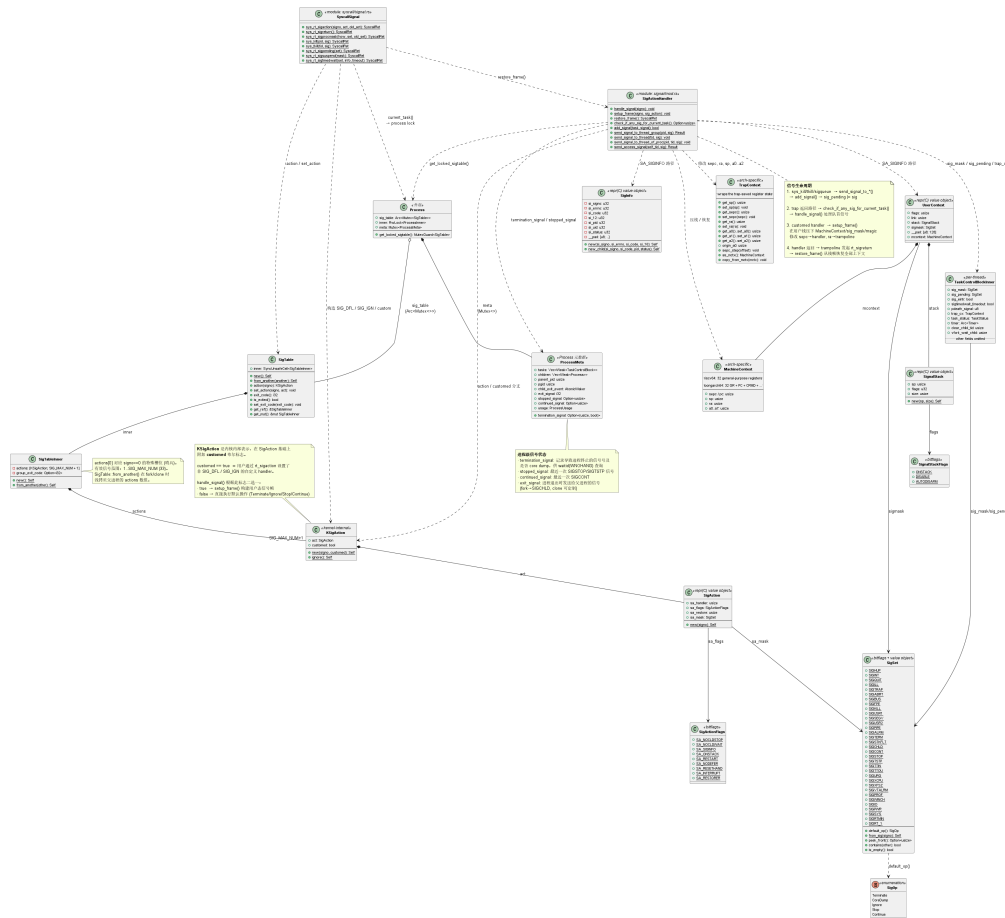


图 4 信号机制核心结构

信号检查发生在从内核态返回用户态之前。若存在 pending & !mask, 内核根据信号动作选择忽略、终止、停止/继续或构建用户态信号帧。自定义 handler 通过修改 TrapContext 跳转到用户函数，并在用户栈保存原始上下文；rt_sigreturn 负责恢复上下文和旧 mask。

Ya2yOS 已处理多个对 Linux 兼容性影响较大的细节：SIGCHLD、WUNTRACED、WCONTINUED、CLD_KILLED、CLD_DUMPED、SA_SIGINFO、SA_RESTART、clear_child_tid 与 futex 唤醒竞态等。

6 文件系统

文件系统以 VFS 作为统一抽象，将底层 ext4、普通文件、设备文件、管道、socket、epoll/eventfd/inotify/mqueue 和挂载上下文统一纳入 fd 模型。底层磁盘文件系统通过 lwext4_rust 绑定 lwext4，在 VirtIO 块设备上提供真实 ext4 读写。

6.1 VFS 分层

抽象	职责
SuperBlock	表示文件系统实例，提供根 inode、同步和 statfs 信息。
Inode	表示目录树节点，负责路径查找、创建、读写、目录项、链接和元数据。
File	表示 fd 可操作对象，统一 read/write/lseek/ioctl/poll/register 等接口。
FdTable	进程级 fd 表，管理 FileDescriptor、close-on-exec、dup 和资源限制。

抽象	职责
FSInfo	保存 cwd、exe、fd 到路径映射和 umask。

6.2 ext4 集成与兼容对象

Ext4Inode 封装 lwext4 文件对象，支持普通文件、目录、软硬链接、rename、truncate、stat、getdents64 和时间戳设置。系统调用层对大块用户缓冲进行分片，避免单次分配导致内核堆压力。

特殊文件通过 File trait 接入：/dev/null、/dev/zero、/dev/rtc、/dev/tty、loop 设备、pipe、eventfd、epoll、inotify、mqueue 等都可以被 fd 表统一管理。/proc 目前偏兼容实现，按需刷新 /proc/<pid>、maps、status、mounts 等内容。

7 网络模块

网络模块位于 os/src/net/ 与 os/src/syscall/net/，受 feature = "net" 控制。它以 smoltcp 为 TCP/IP 状态机，结合文件描述符、任务阻塞和 VirtIO-net 设备层实现 Linux socket API 的主要兼容面。

```
syscall/net
socket, bind, listen, accept, connect, sendmsg, recvmsg
net/socket.rs + tcp.rs + udp.rs + unix.rs
SocketOps, TCP, UDP, Unix domain socket
service.rs + router.rs + wrapper.rs
smoltcp Interface, SocketSet, routing, polling
drivers/virtio/net.rs
LoopbackDevice, EthernetDevice, VirtIO-net
```

TCP/UDP socket 的协议状态保存在全局 SocketSet，具体 fd 对象持有 SocketHandle 和通用选项。Service::poll() 驱动收包、TCP 状态机、重传和发包队列。路由器同时接入 loopback 与 VirtIO-net，使用最长前缀匹配选择出口设备。

Unix domain socket 不经过 smoltcp，而是使用内核队列实现 stream/dgram 和 socketpair()。socket fd 通过 FileClass::Socket 纳入统一 fd 表，因此可参与 read/write、poll、epoll 和非阻塞模式。

8 IO 设备

设备层主要服务块设备和网络设备。基础 BaseDriver 描述设备名称、类型和中断号；BlockDriver 和 NetDriverOps 分别定义块读写与网络收发接口。VirtIO 是当前主要设备框架：

- RISC-V64 使用 MMIO transport，块设备和网络设备位于固定 MMIO 区域。
- LoongArch64 使用 PCI transport，通过 PCI 枚举识别 VirtIO 设备并配置 BAR。
- VirtIoHalCMAImpl 提供 DMA 所需的连续物理页分配、地址转换和 share/unshare。

块设备通过 Disk 光标抽象向 ext4 提供按字节读写。网络设备通过 NetDriverOps 接入 EthernetDevice，再由网络路由器与 smoltcp 交互。当前 IO 路径整体偏同步和轮询，后续可进一步接入中断驱动、waker 和页缓存。

9 项目特色与边界

9.1 特色

- 真实 ext4 后端：比教学文件系统更接近 Linux 用户态预期，可直接读写 ext4 镜像中的用户程序和库。
- 兼容性驱动演进：围绕 glibc、musl、BusyBox、libc-test、LTP 和实际日志持续修补语义。
- 统一 fd 模型：普通文件、socket、pipe、eventfd、epoll、inotify、mqueue、设备文件和挂载上下文共享 fd 表。

- 双架构支持：同一内核代码支持 RISC-V64 与 LoongArch64，设备侧分别适配 MMIO 和 PCI VirtIO。
- Rust 工程化：大量使用 Arc、Weak、锁、trait 和 enum 管理复杂生命周期，同时将 unsafe 控制在页表、DMA、FFI 和用户指针等必要区域。

9.2 当前边界

方向	现状与限制
页缓存	普通文件 IO 与 mmap 尚未通过统一 page cache 贯通，一致性和性能仍有提升空间。
挂载语义	挂载表主要服务 syscall 兼容和 /proc/mounts，尚未形成完整多 superbblock 挂载树。
网络 IO	TCP/UDP 依赖周期性 poll_interfaces()，VirtIO-net 中断与 socket waker 仍需完善。
stub 接口	部分低频或探测型 syscall 采用兼容返回，仍需分类补真实语义。
调度与多核	基础多核路径可运行，但抢占、负载均衡和公平调度仍待增强。

10 总结与展望

Ya2yOS 已从教学内核雏形推进到能够支撑真实 Linux 用户态负载的实验系统。其核心路径已经成形：进程、内存、文件、网络、信号和设备围绕 Linux ABI 协同工作，并通过双架构适配验证了架构抽象的可移植性。

后续重点应从“增加接口数量”转向“做深已有兼容面”：建立统一页缓存、完善挂载树和虚拟文件系统、降低网络轮询开销、补齐真实 loop 设备、梳理兼容 stub、扩大 RISC-V64/LoongArch64 与 glibc/musl 的自动化测试矩阵。